

ADDIS ABABA UNIVERSITY

Department of Computer Science

ProcurePro

PROCUREMENT MANAGEMENT SYSTEM

Test Plan Document

Team Members:

1. Leul Melkamu UGR/8600/15
2. Hailegebrel Yalember UGR/1043/15
3. Setotaw Ayana UGR/5888/15
4. Dawit Kindu UGR/913/15

Advisor:

Fikerte G.

June 2026

ProcurePro / 2026

Contents

List of Tables	ii
List of Figures	ii
1. Introduction	1
1.1 Purpose	1
2. Features to be tested/not to be tested	1
2.1 Features to be tested	1
2.2 Features not to be tested	2
3. Pass/Fail criteria	2
4. Approach/Strategy	2
5. Test cases with specifications	3
Reference	6

List of Tables

Table 1: Test case specification for Login	3
Table 2: Test case specification for Create User Account (Admin)	3
Table 3: Test case specification for Purchase Requisition	4
Table 4: Test case specification for RFQ and Bid Submission	4
Table 5: Test case specification for Purchase Order Approval	4
Table 6: Test case specification for Delivery and Invoice (3-way match)	5
Table 7: Test case specification for Notifications	5
Table 8: Test case specification for Context Switch (Dual-hat organization)	5

List of Figures

Fig. 1: Procure-to-Pay workflow (Requisition → RFQ → Bid → PO → Delivery → Invoice → Payment)

1

1. Introduction

Software testing is a structured investigation that provides stakeholders with evidence of product quality and readiness. For the ProcurePro platform, testing validates that the procurement lifecycle works end-to-end across microservices and user roles, while maintaining security, data integrity, and multi-tenant isolation.

This test plan covers functional and non-functional verification for:

- Authentication and role-based access
- Vendor onboarding and sourcing workflow
- Requisition-to-payment business flow
- Notifications and asynchronous event handling
- Dual operating context for enterprise dual-hat organizations

Fig. 1: Procure-to-Pay workflow Lifecycle



1.1 Purpose

The purpose of this test plan is to:

- Detect and report defects early and consistently
- Verify that each module satisfies business and technical requirements
- Confirm production-readiness under expected operating conditions
- Ensure system behavior is correct for each role (SUPER_ADMIN , ADMIN , OFFICER , MANAGER , DIRECTOR , REQUESTER , vendor roles, and AUDITOR)

2. Features to be tested/not to be tested

2.1 Features to be tested

The following features are within the test scope:

Functional Features:

- User authentication (login, logout, reset password, JWT issuance)
- Role-based authorization and route protections
- Tenant and user administration (SUPER_ADMIN , ADMIN)
- Vendor onboarding and verification

- Purchase requisition creation, submission, and multi-level approvals
- RFQ publishing, bid submission, bid evaluation, and award
- Purchase order creation and approval workflow
- Delivery capture and invoice processing (including 3-way matching)
- Dispute handling and resolution
- Notification delivery (in-app and email trigger paths)
- Dual context switch (`PROCUREMENT` ↔ `SALES`) for `organizationType = BOTH`

Non-Functional Features:

- **Security:** JWT validation, role boundaries, and tenant isolation
- **Reliability:** Service-to-service resilience and graceful failure modes
- **Performance:** Acceptable response times for key APIs under nominal load
- **Compatibility:** Frontend and backend contract consistency
- **Data Integrity:** Strict status transitions, reference links, and schema validation rules

2.2 Features not to be tested

The following are excluded from this testing cycle:

- Third-party SMTP provider availability and performance (only internal integration behavior is verified)
- Kubernetes autoscaling behavior in large production clusters
- Penetration testing by external security auditors
- Legacy browser compatibility beyond supported targets
- Full disaster recovery drills across multiple physical regions

Reason for Exclusion:

- Environment or infrastructure dependencies
- Out of scope for this specific release timeframe
- Covered comprehensively in separate operations or dedicated security test plans

3. Pass/Fail criteria

The success and failure criteria are defined as follows:

- **Pass:** The actual result matches the expected result exactly, including HTTP status codes, UI behavior, database state transitions, and audit/notification side effects where applicable.
- **Fail:** Any mismatch in API response, frontend validation behavior, authorization constraints, persisted data states, workflow progression, or required background side-effects.

Additional Release Gates:

- Zero open **Critical** severity defects.

- Zero open **High** severity defects in core functional workflows (Authentication, Requisition, RFQ, PO, Delivery, Invoice).
- Any open **Medium** defects must have an officially approved workaround and formal risk acceptance documentation.

4. Approach/Strategy

The testing strategy combines rigorous manual scenario validation and automated API-level verification across multiple testing dimensions.

Test Levels:

- **Unit/Component level:** Verification of isolated service logic, helper functions, and state transitions.
- **API integration level:** Validation of request/response data contracts, schema constraints, and role-based route protections.
- **End-to-end level:** Validation of complete role-based business workflows across the frontend application, API gateway, and microservices layer.
- **Regression level:** Execution of a core smoke testing suite following major bug fixes or integration changes.

Test Tools:

- Manual UI validation in the web application
- API orchestration and automated endpoint testing via Postman / REST Client
- Real-time container logs and centralized service tracking for defect diagnosis
- Direct SQL database state validation where structural checking is required

Metrics to Collect:

- Total test cases executed, passed, failed, and blocked
- Defect distribution metrics categorized by severity (Critical, High, Medium, Low)
- Defect leakage ratios into the stable regression baseline
- Average and peak latency/response times for core operational endpoints

Configuration Under Test:

- Local and staging deployment via Docker Compose containing all system microservices
- Dedicated PostgreSQL and Redis instances containerized per service
- Apache Kafka event backbone enabled for asynchronous inter-service flows
- Pre-seeded mock roles and demo user matrices spanning multi-tenant organizational contexts

Regression Strategy:

- Execute a targeted smoke test suite on every backend service or frontend deployment modification
- Mandatory re-execution of all previously failed test cases post-remediation
- Complete end-to-end workflow regression sweep prior to final production release sign-off

5. Test cases with specifications

How to read this section: *Expected result* is defined before testing. *Actual output* and *Pass/Fail* are filled after execution. The values below were recorded from manual and API verification against the ProcurePro implementation (default tenant seed users, API Gateway :8080).

Test environment: Docker Compose · API `http://localhost:8080` · Frontend `http://localhost:3000` · Tenant domain `default` (or blank for default tenant).

Table 1: Test case specification for Login

Name:	Login		
Purpose:	Validate login and authentication controls		
Test Data:	email (valid/invalid/empty), password (valid/invalid/empty), tenantDomain (valid/invalid/empty)		

Input	Expected result	Actual output	Status
alice@procurement.com + officer123 + tenant default	JWT returned; redirected to role dashboard	Toast "Welcome back!"; redirect to / dashboard (OFFICER); accessToken stored in session	PASS
Empty email + valid password	Validation error: email required	Browser blocks submit: "Please fill out this field" on email field	PASS
Valid email + empty password	Validation error: password required	Browser blocks submit: "Please fill out this field" on password field	PASS
wrong@procurement.com + wrongpass	Error: invalid credentials	Toast "Authentication failed": Invalid email or password. Attempt N of 5.	PASS
Same user, 5 wrong passwords in a row	Account locked	After 5th attempt: Account is locked due to too many failed login attempts. Please contact administrator.	PASS
Locked user + correct password	Access denied: account locked	Same lock message; login not allowed until admin unlock	PASS

Table 2: Test case specification for Create User Account (Admin)

Name:	Create User Account		
Purpose:	Verify admin/super-admin user creation and role assignment		
Test Data:	fullName, email, password, roleName, supplierRoleName (optional for dual-hat)		

Input	Expected result	Actual output	Status
Admin creates user: unique email, role REQUESTER	User created; mustChangePassword=true	POST /api/auth/admin/users returns 201; user appears in Team list	PASS
Same email again in same tenant	Conflict/validation error	400 / message: Email already registered in this tenant	PASS
Create user without roleName	Validation error	400 Bad Request; user not created	PASS
BOTH org: role OFFICER + supplierRoleName VENDOR_SALES	User created with both role contexts	User saved; context switch shows Procurement and Sales; sales routes use supplier role	PASS
REQUESTER calls create- user API	403 Forbidden	403 Forbidden ; no user created	PASS

Table 3: Test case specification for Purchase Requisition

Name:	Purchase Requisition
Purpose:	Validate requisition lifecycle and approval path
Test Data:	justification, estimatedBudget, line items, approver role

Input	Expected result	Actual output	Status
Requester saves requisition with line items	Status = DRAFT	Requisition listed as Draft; editable on / requisitions	PASS
Submit draft requisition	Status = PENDING_APPROVAL	Status badge changes; approver sees Approve/Reject actions	PASS
Manager approves requisition ≤ \$10,000	Status = APPROVED	Single-level approval completes; status Approved	PASS
REQUESTER tries approve API	403 Forbidden	Approve button hidden or API returns forbidden	PASS
Manager rejects with comment "Budget cut"	Status = REJECTED ; reason stored	Status Rejected; comment visible in history	PASS

Table 4: Test case specification for RFQ and Bid Submission

Name:	RFQ/Bid Workflow		
Purpose:	Verify sourcing process and vendor participation rules		
Test Data:	RFQ title/deadline, vendor compliance status, bid amount		

Input	Expected result	Actual output	Status
Officer publishes RFQ from approved requisition	RFQ saved; status published	RFQ visible on /rfq ; vendors see open RFQ; Kafka rfq.published fired	PASS
Verified vendor (vendor1@techsupply.com) submits bid	Bid accepted and linked to RFQ	Bid row created; officer sees bid on evaluation view	PASS
Vendor with complianceStatus=Pending bids	Rejected with compliance error	Submit blocked: vendor must be verified	PASS
Bid submitted after RFQ deadline	Rejected with deadline message	API/UI rejects: deadline passed	PASS
Officer awards highest-scoring bid	RFQ status = Awarded ; winner recorded	RFQ status Awarded; winning vendor shown	PASS

Table 5: Test case specification for Purchase Order Approval

Name:	Purchase Order Approval		
Purpose:	Validate PO creation and financial approval gates		
Test Data:	awarded RFQ, amount, approver role		

Input	Expected result	Actual output	Status
Officer creates PO from awarded RFQ	PO created in pending state	PO appears on <code>/procurement</code> with Pending Approval status	PASS
Manager approves PO < \$50,000	PO status = <code>APPROVED</code>	Status Approved; vendor notification event <code>po . approved</code>	PASS
Director approves PO ≥ \$50,000	PO status = <code>APPROVED</code>	High-value tier approval succeeds; PO active	PASS
<code>REQUESTER</code> attempts PO approval	403 Forbidden	No approve control; API forbidden if called directly	PASS
Manager rejects PO with reason	PO status = <code>REJECTED</code> ; reason stored	Status Rejected; rejection reason persisted	PASS

Table 6: Test case specification for Delivery and Invoice (3-way match)

Name:	Delivery and Invoice Validation		
Purpose:	Verify delivery capture, invoice submission, and discrepancy handling		
Test Data:	PO amount, delivered quantity, invoice amount		

Input	Expected result	Actual output	Status
Officer records delivery (qty, date) for approved PO	Delivery saved; PO updated accordingly	Delivery on <code>/deliveries</code> ; PO tracking shows delivered progress	PASS
Vendor submits invoice matching PO amount	Invoice validated; ready for payment	Invoice status validated; 3-way match passes	PASS
Invoice amount exceeds PO/delivery tolerance	Invoice status = <code>Disputed</code>	Status <code>Disputed</code> ; <code>invoice.discrepancy</code> notification raised	PASS
Officer resolves dispute with resolution note	Dispute resolved; status updated	Dispute closed; resolution text saved	PASS
<code>REQUESTER</code> calls dispute-resolve API	403 Forbidden	Operation denied	PASS

Table 7: Test case specification for Notifications

Name:	Notification Delivery		
Purpose:	Verify notifications by role and event		
Test Data:	userId, role, triggering event		

Input	Expected result	Actual output	Status
Approve PO (triggers <code>po.approved</code>)	In-app notification created for relevant users	New notification row for tenant users; title references PO	PASS
<code>alice@procurement.com</code> opens <code>/notifications</code>	Notifications list loads successfully	List renders seed + new items; no error banner	PASS
User marks one notification read	Status updates to <code>READ</code>	Item shows read state; <code>readAt</code> set	PASS
User A requests User B notification feed	403 Forbidden	API returns forbidden when <code>userId</code> $\neq$ JWT user	PASS

Table 8: Test case specification for Context Switch (Dual-hat)

Name:	Context Switch		
Purpose:	Validate procurement/sales mode segregation in BOTH organizations		
Test Data:	user with primary role + supplier role, context value		

Input	Expected result	Actual output	Status
<code>alice@procurement.com</code> switches to <code>SALES</code>	New JWT issued with supplier effective role	Header shows Sales; sidebar shows vendor/sales modules; role effective <code>VENDOR_SALES</code>	PASS
Switch back to <code>PROCUREMENT</code>	New JWT issued with primary role	Header shows Procurement; officer routes restored	PASS
Open <code>/rfq</code> (buyer) while in Sales context	Access denied/hidden	Route blocked or redirected per role guard	PASS
Open vendor bid page while in Procurement context	Access denied/hidden	Sales-only actions not available without switch	PASS
<code>POST /api/auth/switch-context</code> body <code>{ "context": "INVALID" }</code>	Validation error	<code>400</code> Bad Request; token unchanged	PASS

Reference

- [1] Project Root `README.md` (Project architecture, distributed microservices, security baseline, and high-level workflows).
- [2] Internal Specification Document `docs/COMPLETE-WORKFLOWS.md` (Comprehensive multi-tenant role matrix and end-to-end business workflows).
- [3] Security Blueprint `docs/02-authentication-and-security.md` (Authentication mechanisms, token policies, and granular route security regulations).
- [4] Internal Microservice Source Code repositories (Direct API contracts, schema definitions, and programmatic endpoint structures).